

```

    }
    private static final long serialVersionUID = 1L;
    private String code = null;
    public String getCode() {
        return code;
    }
    public void setCode(String code) {
        if(code == null) {
            this.code = null;
        } else {
            this.code = code;
        }
    }
    private String message = null;
    public String getMessage() {
        return message;
    }
    public void setMessage(String message) {
        if(message == null) {
            this.message = null;
        } else {
            this.message = message;
        }
    }
    public Object clone() {
        MessageInfo copyObj = new MessageInfo();
        copyObj.clone(this);
        return copyObj;
    }
    public void clone(DataObject _messageInfo) {
        if(this == _messageInfo)
            return;
        MessageInfo _messageInfo = (MessageInfo) _messageInfo;
        this.setCode(_messageInfo.getCode());
        this.setMessage(_messageInfo.getMessage());
    }
    public String toString() {
        StringBuilder buffer = new StringBuilder();
        buffer.append("code : ").append(code).append("\n");
        buffer.append("message : ").append(message).append("\n");
        return buffer.toString();
    }
    private static final Map<String, FieldProperty> fieldPropertyMap;
    static {

```

```
fieldPropertyMap.put("code", FieldProperty.builder()
    .setPhysicalName("코드")
    .setLogicalName("코드")
    .setType(FieldProperty.Type.OBJECT_STRING)
    .setDecimal(-1)
    .setNullable(true)
    .setEncrypted(false)
    .build());
fieldPropertyMap.put("message", FieldProperty.builder()
    .setPhysicalName("message")
    .setLogicalName("메시지 내용")
    .setType(FieldProperty.Type.OBJECT_STRING)
    .setDecimal(-1)
    .setNullable(true)
    .setEncrypted(false)
    .build());
}
```

```
public Map<String, FieldProperty> getFieldPropertyMap() {
    return Collections.unmodifiableMap(fieldPropertyMap);
}
```

```

/*****
 * Copyright 2026 by Nonghyup. All rights reserved. Nonghyup 의 사전 승인 없이
 * 본 내용의 전부 또는 일부를 복사, 배포, 사용을 금합니다. Nonghyup의 사전 승인
 * 없이 소스코드를 변경하여 사용하는 경우 소스코드에 대한 물리적 과 책임을 보장하지 않습니다.
 *
 * If you modify this source without Nonghyup's approval, Nonghyup does
 * not guarantee the quality and performance of source.
 *****/
/*****
 * 이 력 사 후
 *****/
/*****
 * 원 자 비 전 작 성 자 변 경 사 항
 *****/
/*****
 * 2026. 7. 21. 1.0 종 권 동 최 초 작 성
 *****/
package nhnis.fw.commons.message;
import javax.sql.DataSource;
import org.springframework.context.ApplicationContext;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import jakarta.annotation.PostConstruct;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
/
```

```

/
import jakarta.annotation.PostConstruct;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
/
<PRE>
*
*
</PRE>
*
*
@author 종 권 동
*
@version 1.0, 2026. 7. 21.
*
@logicalName
/
```

```

@Sif4j
@Component
@RequiredArgsConstructor
public class MessageLoader {

    private final ApplicationContext applicationContext;
    private final MessageProperties properties;
    private final MessageCache messageCache;

    @PostConstruct
    public void init() {

        if (properties.isEnabled()) {
            log.info("DB Message Bundle Disabled");
            return;
        }

        if (properties.getDataSource() == null || properties.getDataSource().isBlank()) {
            throw new IllegalArgumentException("Message Bundle 데이터 소스 설정이 없습니다.");
        }

        DataSource dataSource = applicationContext.getBean(properties.getDataSource(),
            DataSource.class);
        JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);

        messageCache.clear();
        String tables[] = properties.getTableNames();
        StringBuilder sql = new StringBuilder();
        for (int i = 0; i < tables.length; i++) {
            if (i > 0) {
                sql.append(" UNION ALL ");
            }
            sql.append("SELECT CRM_MSG_C, MSG_CNTN FROM ");
            sql.append(tables[i]);
        }
        jdbcTemplate.query(
            sql.toString(),
            rs -> {
                while (rs.next()) {
                    messageCache.put(
                        rs.getString("CRM_MSG_C"),
                        rs.getString("MSG_CNTN"));
                }
            }
        );

        log.info("DB Message Bundle Loaded: 총 {}건 조회", messageCache.size());
    }
}

/*****
 * Copyright 2026 by Nonghyup. All rights reserved. Nonghyup의 사전 없이
 * 본 내용의 전부 또는 일부를 복사, 배포, 사용을 금합니다. Nonghyup의 사전 승인
 * 없이 소스코드를 변경하여 사용하는 경우 소스코드에 대한 물리적, 법적 책임을 보증하지 않습니다.
 * If you modify this source without Nonghyup's approval, Nonghyup does
 * not guarantee the quality and performance of source.
 *****/

```



```

import org.springframework.core.MethodParameter;
import org.springframework.stereotype.Component;
import org.springframework.web.bind.support.WebDataBinderFactory;
import org.springframework.web.context.request.NativeWebRequest;
import org.springframework.web.context.request.RequestContextHolder;
import org.springframework.web.context.request.ServletRequestAttributes;
import org.springframework.web.method.support.HandlerMethodArgumentResolver;
import org.springframework.web.method.support.ModelAndViewContainer;
import com.fasterxml.jackson.databind.DeserializationFeature;

import java.util.stream.Collectors;

package nhnis.fw.commons.resolver;
}

@Target(ElementType.PARAMETER)
@Retention(RetentionPolicy.RUNTIME)
public @interface RequestBody {

import java.lang.annotation.Annotation;
import java.lang.annotation.Retention;
import java.lang.annotation.Target;

package nhnis.fw.commons.resolver;
}

private boolean enabled = false;
private String dataSource;
private String[] tableName;

public class MessageProperties {
    @ConfigurationProperties(prefix="framework.message")
    @Component
    @Setter
    @Getter
    /*
    * @author 홍길동
    * @version 1.0, 2026. 7. 21.
    * @logicalName
    */
    /*
    * <PRE>
    * 설명
    * </PRE>
    */
import lombok.Getter;
import lombok.Setter;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

package nhnis.fw.commons.message;
}

/*****
* 이력 사항
*
* 립자변경사항
*
* 2026. 7. 21. 1.0 홍길동 최초작성
*****/

```

```
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.servlet.http.HttpServletRequest;
import lombok.RequiredArgsConstructor;
```

```
@Component
@RequiredArgsConstructor
public class RequestBodyArgumentResolver implements HandlerMethodArgumentResolver {
```

```
private final ObjectMapper objectMapper;
private static final String MULTI_PART = "multipart/";
```

```
@Override
public boolean supportsParameter(MethodParameter parameter) {
    HttpServletRequest request =
        ((ServletRequestAttributes)RequestContextHolder.getRequest()).getRequest();
    String contentType = request.getContentType();
    if (contentType != null && contentType.toLowerCase().startsWith(MULTI_PART)) {
        return false;
    }
}
```

```
return parameter.hasParameterAnnotation(RequestBody.class);
```

```
@Override
public Object resolveArgument(MethodParameter parameter, ModelAndViewContainer
    mavContainer, NativeWebRequest webRequest, WebDataBinderFactory binderFactory) throws Exception {
    objectMapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
    objectMapper.configure(DeserializationFeature.ACCEPT_EMPTY_ARRAY_AS_NULL_OBJECT,
        true);
```

```
HttpServletRequest request = webRequest.getNativeRequest(HttpServletRequest.class);
String requestBody = request.getReader().lines().collect(Collectors.joining());
```

```
JsonNode root = objectMapper.readTree(requestBody);
JsonNode bodyNode = root.get("dto");
```

```
Class<?> dtoClass = parameter.getParameterType();
```

```
return objectMapper.treeToValue(bodyNode, dtoClass);
}
```

```
}
package nhnis.fw.commons.resolver;
```

```
import java.io.IOException;
import java.text.MessageFormat;
import java.util.Arrays;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.MethodParameter;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.HttpClientConnector;
import org.springframework.http.converter.HttpMessageConverter;
import org.springframework.http.server.ServerHttpRequest;
import org.springframework.http.server.ServerHttpResponse;
import org.springframework.web.bind.annotation.ControllerAdvice;
```



```

import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.servlet.mvc.annotation.AnnotationMethodHandler;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.node.ObjectNode;
import com.tmax.superspring.dto.engine.exception.MarshalException;
import nhn.fw.commons.configuration.WebConfiguration;
import nhn.fw.commons.context.ServiceContextHolder;
import nhn.fw.commons.context.SpringContext;
import nhn.fw.commons.dto.NH_NIS_ERR_DTO;
import nhn.fw.commons.dto.NH_NIS_ERR_DTO_MsgJson;
import nhn.fw.commons.exception.NhBaseException;
import nhn.fw.commons.exception.NhBaseExceptionType;
import nhn.fw.commons.message.MessageCache;

```

```

@ControllerAdvice
public class ResponseBodyArgumentResolver implements ResponseBodyAdvice<Object> {

```

```

    private final WebConfiguration webConfiguration;

```

```

    private final ClientHttpRequestConnector connector;

```

```

    @Autowired

```

```

    private MessageCache messageCache;

```

```

    private final ObjectMapper objectMapper = new ObjectMapper();

```

```

    private static final String PREFIX
    private static final String HEADER
    private static final String DTO
    private static final String DTO_LOGICAL_NAME
    private static final String FIELD_PROPERTY_MAP
    private static final String SYS_COMM
    private static final String MULTIPART
    private static final String MULTIPART
    private static final String MULTIPART

```

```

    ResponseBodyArgumentResolver(ClientHttpRequestConnector connector, WebConfiguration
webConfiguration) {

```

```

        this.connector = connector;

```

```

        this.webConfiguration = webConfiguration;
    }

```

```

    @Override
    public boolean supports(MethodParameter returnType, Class<?> extends HttpMessageConverter<?>
converterType) {

```

```

        return true;
    }

```

```

    @Override
    public Object beforeBodyWrite(Object body, MethodParameter returnType, MediaType
selectedContentType, Class<?> extends HttpMessageConverter<?> converterType, ServerHttpRequest request,
ServerHttpResponse response) {

```

```

        ObjectNode responseBody = objectMapper.createObjectNode();

```

```

        try {

```

```

            if (request.getHeaders().getContentType().toString().startsWith(MULTIPART)) {

```

```

                return body;
            }

```

```

        }
        /* Response Header */

```

```

        if (ServiceContextHolder.getInstance() != null) {

```

```

            ObjectNode headerNode =

```